System design package

Four complementary views of an AI-assisted network intrusion detection system

Scope note: The supplied Module 1 notebook covers offline GeNIS data ingestion, preprocessing, class reporting, and Random Forest baseline evaluation. Live traffic capture, inference service, alerting, and dashboard are shown as proposed system-level components; include them in the final design only if they are in your project scope.

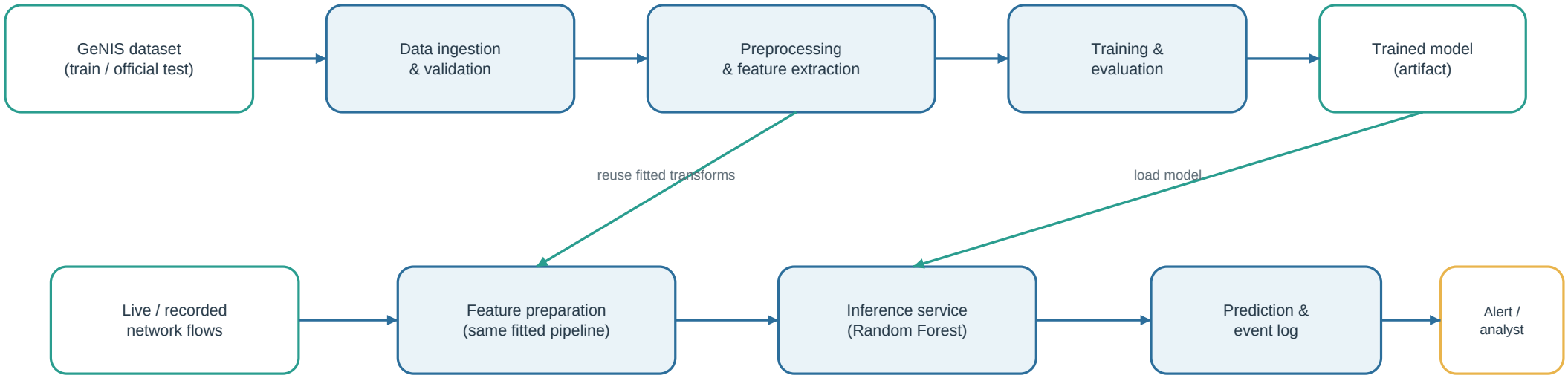
Selected designs and their purpose

Component diagram	Defines system modules and their connections.
Data Flow Diagram	Traces traffic and prediction data between processes and stores.
Activity diagram	Shows the decision workflow from input validation to classification.
Sequence diagram	Shows message order between runtime components for one flow.

Design assumptions are stated in each diagram; replace proposed components with the actual implementation as it evolves.

1. Component Diagram

Structural design | Major components and dependencies



Why this design was chosen

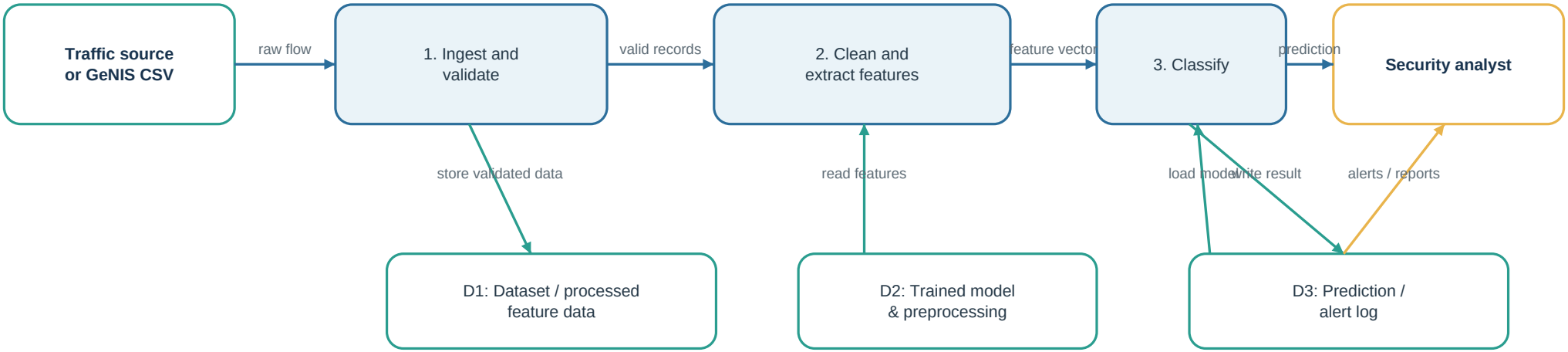
A component diagram is chosen because it gives reviewers a quick map of responsibilities and interfaces. It separates the offline model-building pipeline from the proposed runtime detector, helping clarify which parts already exist and which are planned.

How it fits Module 1

Module 1 directly implements the upper training path through model evaluation. The lower runtime path is a system-level proposal and depends on the eventual deployment scope.

2. Data Flow Diagram

Behavioral / information design | Inputs, processes, stores, outputs



Why this design was chosen

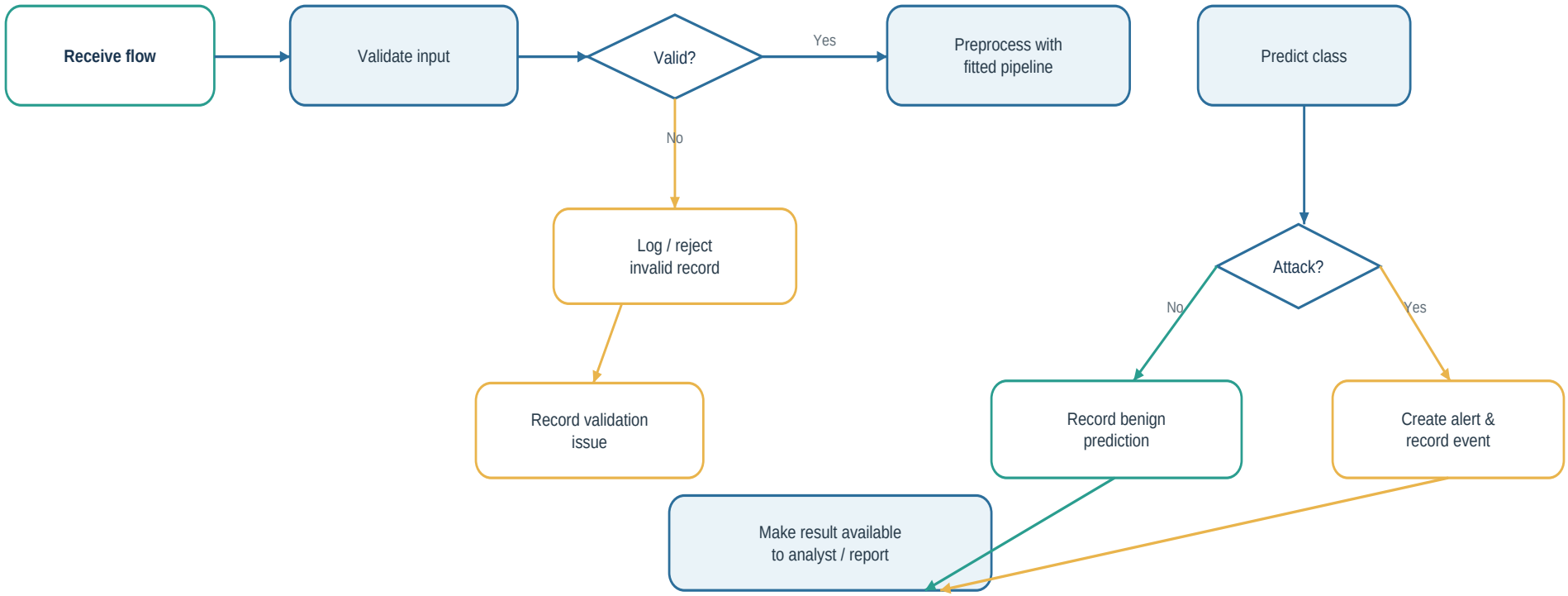
The DFD is chosen to make data boundaries visible: where source records enter, what transformations occur, which artifacts are stored, and who consumes the results. This is especially useful for discussing label isolation, training-only fitted transforms, and keeping the official test partition separate.

How it fits Module 1

Module 1 provides the GeNIS CSV input, ingestion, cleaning, feature preparation, training, and evaluation. D3 and analyst-facing alerts are proposed runtime outputs unless implemented elsewhere.

3. Activity Diagram

Behavioral design | Detection workflow and decision paths



Why this design was chosen

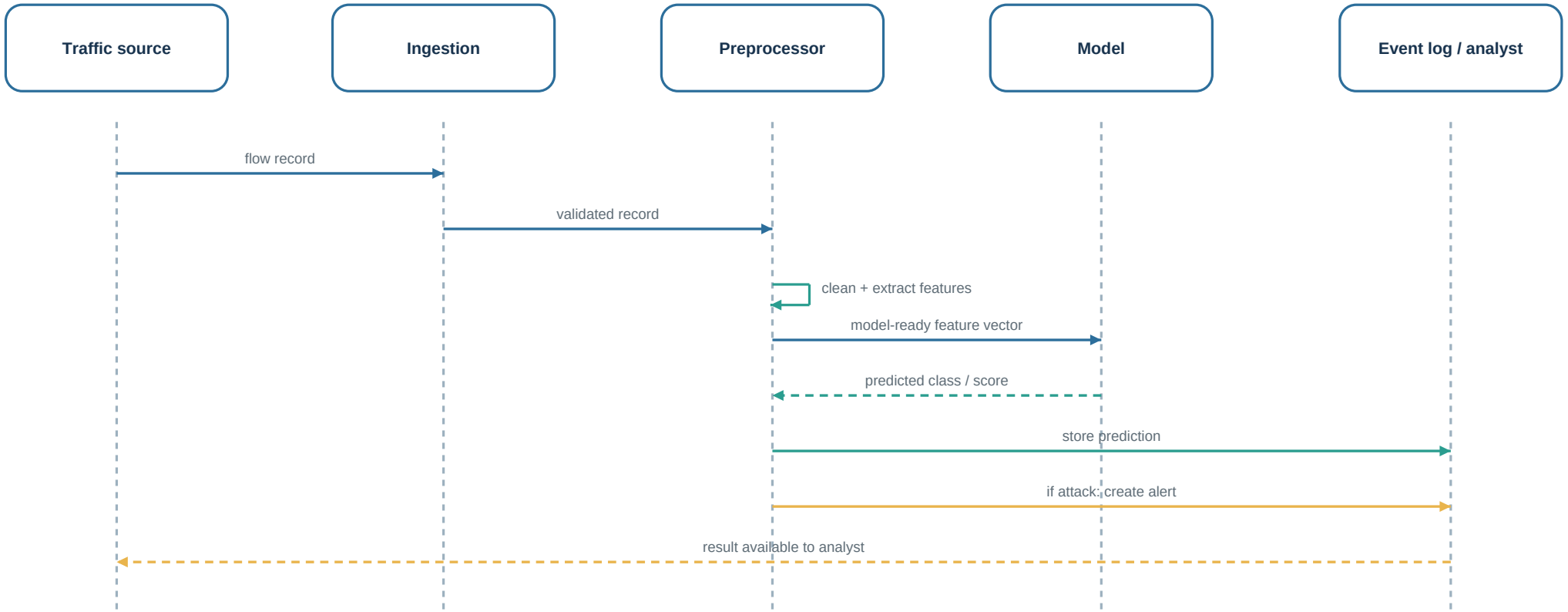
The activity diagram is chosen because it explains control logic, not just data movement. It makes validation failures and the attack-versus-benign decision explicit, which helps reviewers understand the detector’s behavior and exception path.

How it fits Module 1

The notebook runs the offline preprocessing and classification steps; input streaming, alert creation, and analyst presentation are design-level extensions.

4. Sequence Diagram

Behavioral design | Message order for one flow



Why this design was chosen

The sequence diagram is chosen to clarify timing and responsibility across components. It shows that the preprocessor applies the same fitted transformations before inference, that the model returns a prediction, and that alert logging follows only when the prediction indicates an attack.

How it fits Module 1

The model-training/evaluation notebook does not implement these runtime interactions; this sequence describes the proposed inference path when the model is deployed.